

Minimum Viable Kernel (MVK)

Collaboration & Reproducibility Proposal

The Minimum Viable Kernel (MVK) project has successfully demonstrated that it is possible to mathematically eliminate spatial and temporal memory vulnerabilities in a bare-metal architecture using Rust and Lean 4 formal verification. As we prepare for the **v9.0 Security Hardening Release**, we are officially opening the repository for community collaboration, peer review, and academic reproduction.

1. How to Reproduce the MVK Empirical Results

We strongly encourage independent security researchers and systems engineers to reproduce our benchmarks and formal verification proofs. All infrastructure relies on open-source toolchains.

Prerequisites:

- Rust Toolchain: nightly-2026-05-15 (required for no_std and Miri).
- Lean 4: v4.29.1 (for formal verification proofs).
- Hardware: An x86_64 virtualization environment (QEMU) or a bare-metal instance.

Reproduction Steps:

1. Clone the Repository:

```
git clone https://github.com/xaviercallens/rust-linux-mini-kernel.git
```

2. Verify the Lean 4 Mathematical Proofs:

```
cd specs/lean4
lake build
```

3. Run the Dynamic Undefined Behavior (UB) Interpreter:

```
cargo miri test
```

4. Execute the Bare-Metal Kernel in QEMU:

```
cargo run --release
```

2. Proposed Improvements & Next Steps (v9.0)

The foundation built on Rust's Ownership Model provides absolute temporal and spatial safety. However, the architectural design must evolve to mitigate logical flaws and hardware-level exploitation.

A. Hardware-Boundary Fuzzing (libFuzzer)

We need contributors to write rigorous cargo-fuzz targets to stress-test the entry and exit points of unsafe abstractions.

B. Microkernel Privilege De-escalation

Refactor device drivers to execute in Ring-3 user space. We invite OS architects to help design a high-throughput, zero-copy IPC mechanism utilizing our new RcuPointer.

C. Advanced Exploit Mitigations

Implement Kernel Address Space Layout Randomization (KASLR) and Kernel Control-Flow Integrity (kCFI).

D. Extending Lean 4 Formal Verification

We need mathematical logicians to help write Lean 4 algebraic models for the Virtual File System (VFS) and the asynchronous packet ingress pathways.

3. How to Contribute

1. Review the open issues labeled *good first issue* or *security*.
2. Fork the repository and create a feature branch.
3. Ensure your branch passes the GitHub Actions CI/CD pipeline (Miri and cargo-audit).
4. Submit a Pull Request.

We look forward to collaborating with you to build the most secure operating system in the world.